

DevSecOps & Platform Engineering — A Beginner’s Training Log

Built inside the poetic-musings repository

James Clements III

September 19, 2026

Contents

Why this document exists	2
Part 0 — The big picture: what is “DevSecOps” actually?	2
Part 1 — Infrastructure as Code, with Terraform	3
1.1 What problem does this solve?	3
1.2 What was built: <code>infra/terraform/</code>	3
1.3 What “hardening” actually means here	5
1.4 What was, and wasn’t, verified	6
Part 2 — Containers and Kubernetes	6
2.1 What problem do containers solve?	6
2.2 What was built: a real, working service	6
2.3 The Dockerfile: hardening a container image	7
2.4 The Kubernetes manifests: <code>k8s/</code>	7
2.5 What was, and wasn’t, verified	9
Part 3 — CI/CD pipelines with security scanning built in	9
3.1 What problem does this solve?	9
3.2 What was built: <code>.github/workflows/</code>	10
3.3 Walking through the security stages	10
3.4 Pipeline design choices worth naming in an interview	11
3.5 The GitLab mirror: <code>.gitlab-ci.yml</code>	11
Part 4 — Where the “sysadmin” and “compliance” pieces of the job fit	12
Part 5 — Honest gaps: what this repo cannot teach you	13
Part 6 — Suggested next exercises, roughly in order of effort	13
Glossary	14

Why this document exists

You’re moving into a Senior DevSecOps / Platform Engineer role, and you wanted a hands-on way to see the pieces of that job actually working, not just read about them. This document is that: a guided tour of a small, real “platform” that got built on top of poetic-musings — an FPGA/hardware/music hobby repository that had *zero* cloud or container footprint before this — specifically to give you something to point at, poke at, and learn from.

It is written slowly and gently on purpose. If a term shows up that a hiring manager would expect you to already know, it gets defined the first time it’s used, in plain language, before the code that uses it. You don’t need prior Terraform, Kubernetes, or CI/CD experience to follow this — you do need patience, because there’s a lot here, and some of the concepts genuinely take a few readings to settle.

How to use this document: read it top to bottom once, without trying to memorize anything. Then go open the actual files it references, side by side with each section, and read them together. The goal isn’t to memorize YAML syntax — it’s to build the mental model of *why* each piece exists, so that when you see it in a real company’s infrastructure, you recognize the shape of it immediately.

At the end there’s a glossary and a set of suggested next exercises.

Part 0 — The big picture: what is “DevSecOps” actually?

Before any code, let’s build the mental model.

DevOps is the practice of *not* having a wall between the people who write software (“Dev”) and the people who run it in production (“Ops”). Historically those were separate teams: developers wrote code and threw it over a wall, operations caught it and had to figure out how to run it, and when something broke, each side blamed the other. DevOps collapses that wall: the same practices, tools, and often the same people, own the whole lifecycle — writing code, testing it, packaging it, deploying it, monitoring it, and fixing it when it breaks. The tooling that makes this possible is **automation**: scripts and pipelines that build, test, and deploy code without a human manually SSHing into a server and running commands by hand.

DevSecOps adds one more piece: security stops being a separate team that reviews your code right before launch (often too late to fix anything cheaply) and instead becomes part of the automated pipeline itself. Every time code is pushed, automated tools check it for secrets accidentally committed, known-vulnerable dependencies, insecure code patterns, and misconfigured infrastructure — *before* a human ever has to remember to ask “is this safe?” This is often summarized as “**shifting left**”: security work moves from the right-hand end of the timeline (a pre-launch audit) toward the left-hand end (the moment you write the code), because catching a mistake in a 30-second automated scan costs nothing, while catching the same mistake after it’s running in production can cost days and a security incident report.

A **Platform Engineer** is the person who builds the *shared* infrastructure that every product team stands on — sometimes called an **Internal Developer Platform (IDP)**. Think of it like this: instead of every team reinventing how to get a container running in the cloud, the platform team builds one well-lit, secure, paved road (standard CI/CD pipelines, standard Kubernetes clusters,

standard scanning tools, standard cloud provisioning patterns), and every other team just drives on it. Senior DevSecOps Engineers on a platform team build and secure *that road*, not just one team's app.

That's the job. Everything below is a small, honest attempt to build a miniature version of that road, inside a repo that had none of it, so you can walk it end to end.

Part 1 — Infrastructure as Code, with Terraform

1.1 What problem does this solve?

Before “Infrastructure as Code” (IaC) was normal practice, provisioning cloud infrastructure — a server, a database, a storage bucket, a network — meant a human clicking through a cloud provider's web console. This has serious problems: it's not repeatable (nobody remembers exactly what they clicked six months ago), it's not reviewable (you can't put a mouse-click history through a code review), it drifts (one environment slowly becomes different from another because of manual tweaks), and it's slow (rebuilding an environment from scratch after a disaster could take days).

Infrastructure as Code means you describe the infrastructure you want in a text file, check that file into version control (git, same as your application code), and a tool reads that file and makes the real cloud match it. Change the file, and the *next* run makes the cloud match the *new* description. This gets you: a full history of every infrastructure change (via `git log`), code review on infrastructure changes exactly like application changes, the ability to recreate an entire environment from nothing by running the tool again, and — critically for DevSecOps — the ability to run *automated security scanners* against your infrastructure definitions before they're ever applied to a real account, the same way you'd scan application code.

Terraform (by HashiCorp) is the most widely used IaC tool. You write files in a language called HCL (HashiCorp Configuration Language) describing “resources” (a VPC, an S3 bucket, a virtual machine), and Terraform figures out the plan to create, update, or destroy real cloud resources to match what you described.

1.2 What was built: `infra/terraform/`

Open this directory alongside reading here. The layout is:

```
infra/terraform/
|-- backend.tf           # where Terraform stores its "state"
|-- providers.tf         # which clouds we're talking to
|-- variables.tf         # inputs to the whole config
|-- outputs.tf           # values Terraform prints out after running
|-- main.tf              # wires the two platform modules together
|-- modules/
|   |-- aws-platform/    # everything for the AWS side
|   |-- azure-platform/  # everything for the Azure side
|-- environments/
|   |-- dev/terraform.tfvars  # dev-specific settings
|   |-- prod/terraform.tfvars # prod-specific settings
-- README.md
```

A **module** in Terraform is a reusable package of resources — like a function in a programming language. Instead of writing “create a VPC, create subnets, create an ECR repo, create an S3 bucket” by hand in every environment, you write that once as a module, and then each environment just calls the module with different settings (**dev** gets 1 replica of something, **prod** gets 3; **dev** might use a smaller instance size, and so on). That’s what `environments/dev/terraform.tfvars` and `environments/prod/terraform.tfvars` are: the *same* module, called twice, with different input values. This is the core IaC lesson: **don’t copy-paste your infrastructure definitions between environments — reuse them with different inputs.**

backend.tf — where does Terraform remember what it already built?

When Terraform creates real infrastructure, it needs to remember what it created, so that next time you run it, it knows the difference between “this resource already exists, leave it alone” and “this is new, create it.” That memory is called **state**, and by default Terraform stores it in a local file on your laptop. That’s dangerous the moment more than one person (or a CI pipeline) needs to run Terraform, because two people running at once with two different local state files will conflict and can corrupt real infrastructure. The fix is a **remote backend**: state gets stored in a shared location (here, an S3 bucket, with a DynamoDB table used purely as a *lock* — a “somebody is already running Terraform right now, wait your turn” flag) so that everyone and every pipeline shares one source of truth, with protection against two runs stepping on each other. `backend.tf` documents this pattern (commented out, since there’s no real S3 bucket to point at yet) and notes the Azure equivalent (Azure Storage Account with blob leasing for locking).

providers.tf — which cloud, which version

A **provider** is Terraform’s plugin for talking to a specific system — **aws** for Amazon Web Services, **azurerm** for Azure. This file declares both, and pins their versions (~> 5.60 for AWS, ~> 3.110 for Azure). Pinning versions matters for the same reason pinning dependency versions matters in application code: an unpinned provider could silently start behaving differently after an upstream update, in a way you didn’t ask for and might not notice until something breaks.

modules/aws-platform/ — walking through it file by file

- **network.tf** — a VPC (Virtual Private Cloud: your own private network inside AWS) with public and private subnets spread across two Availability Zones. Availability Zones are physically separate data centers within a region; spreading resources across two of them means one data center having a bad day doesn’t take your whole environment down. Public subnets can reach the internet directly; private subnets can’t (a deliberate security boundary — anything that doesn’t need to be reachable from the internet shouldn’t be).
- **ecr.tf** — an **ECR** (Elastic Container Registry) repository: this is where Docker container images get stored (more on containers in Part 2). Three security choices worth noticing: it’s **KMS-encrypted** (encrypted at rest using AWS’s Key Management Service, not just “AWS handles it somehow”), it has **scan_on_push** turned on (every image pushed here gets automatically scanned for known vulnerabilities, no human has to remember to trigger it), and it uses **IMMUTABLE** tags — meaning once an image is pushed as **v1.2.3**, nobody can silently overwrite what **v1.2.3** points to. That last one matters more than it sounds: without immutability, an attacker (or a careless teammate) who gets push access could replace a known-good image with a malicious one *without changing its name*, and anything referencing **v1.2.3** would silently start running the new, bad image.

- **s3.tf** — an S3 bucket for storing build artifacts, with **versioning** (deleting or overwriting a file doesn't destroy the old version, it just becomes non-current — this is your safety net against accidental deletes, and also against ransomware-style overwrite attacks), **SSE-KMS encryption** (again, encrypted at rest), and a **public access block** (an explicit setting that makes it structurally impossible for this bucket to be made public, even by a future misconfiguration — this exists because “someone accidentally made an S3 bucket public” is one of the most common real-world cloud breaches). There's also a *second*, separate bucket just for access logs — a small but important pattern: a bucket shouldn't log to itself, because that creates a confusing feedback loop and makes log tampering harder to detect.
- **kms.tf** — the encryption key itself, a **KMS Customer Managed Key (CMK)** with automatic rotation enabled, so the actual encryption key material changes periodically without you having to remember to do it.
- **iam.tf** — an **IAM** (Identity and Access Management) role that a CI pipeline would assume to push images and artifacts. The important lesson here is **least privilege**: this role's policy is scoped to the *exact* ARN (Amazon Resource Name — a unique identifier for one specific resource) of the one ECR repo and one S3 bucket this module created — not **"Resource": "*" (which would mean “any resource in the account”)**. If this role's credentials ever leaked, the blast radius is “can push to one specific registry and one specific bucket,” not “can do anything in this AWS account.”

modules/azure-platform/ — the same ideas, Azure's names for them

- A **resource group** (Azure's container concept for grouping related resources together, similar in spirit to “everything in one VPC” on AWS, but really just an organizational/billing boundary).
- A **Premium ACR** (Azure Container Registry — Azure's equivalent of ECR), with admin access disabled (admin credentials are a shared secret; disabling them forces everyone to authenticate with their own individual, revocable identity instead) and public network access disabled.
- A **GRS storage account** (Geo-Redundant Storage — data is replicated to a second, geographically distant Azure region automatically) with TLS 1.2 minimum (rejects older, weaker encryption protocols), shared-key auth disabled (same reasoning as ACR admin access above — forces per-identity authentication instead of one shared secret anyone with it can use), and blob versioning plus soft-delete (same safety-net reasoning as the S3 versioning above).
- A large comment block documenting how this would connect to an on-premises Windows Active Directory via **Azure AD Connect** — this is *not* implemented (there's no real AD here — see Part 5), but it's documented so you understand where that puzzle piece would slot in.

1.3 What “hardening” actually means here

You'll hear “hardening” a lot in this job. It just means: configuring a system so that its *default* behavior is the safe one, and anything unsafe requires a deliberate, explicit choice — rather than the reverse, where the default is open and you have to remember to lock each thing down. Every choice above is an example: encryption is on by default, not something you have to opt into; public access is blocked by default, not something you have to remember to disable; permissions are scoped tightly by default, not granted broadly “to be safe.” A useful habit for interviews: when you see a cloud resource definition, ask “what's the *default* here if I hadn't touched it, and did someone

deliberately override that default toward safety?”

1.4 What was, and wasn’t, verified

`terraform`, `tfsec` (a Terraform-specific security scanner), and `checkov` (a broader IaC security scanner) are not installed on the machine this was built on, so none of this was run through Terraform’s own validator or an automated scanner locally. It was checked by hand — brace-matching across every file, and correcting one real mistake caught along the way (an Azure provider attribute name, `enable_https_traffic_only`, that changed to `https_traffic_only_enabled` in a newer provider version — the kind of small drift that `terraform validate` would catch instantly, which is exactly why that check belongs in an automated pipeline instead of a human’s memory). The CI/CD pipeline built in Part 3 now runs `tfsec` against this code on every push, which is the honest, scalable way to keep it correct going forward — not a one-time human review.

None of this has been run against a real AWS or Azure account. That’s an important distinction to be able to say out loud in an interview: this demonstrates *how to write* correct, hardened IaC — it does not demonstrate having operated it against production load, IAM federation, or a real incident. That’s a genuine, named gap (see Part 5).

Part 2 — Containers and Kubernetes

2.1 What problem do containers solve?

Before containers, “it works on my machine but not in production” was a constant headache — because “my machine” had a slightly different version of Python, a slightly different set of system libraries, a different OS, than the production server. A **container** solves this by packaging an application together with *everything* it needs to run — the exact Python version, the exact libraries, the exact OS-level dependencies — into one portable image, so it behaves identically wherever it runs: your laptop, a CI pipeline, or a production server.

Docker is the most common tool for building and running containers. You write a `Dockerfile` — a recipe describing how to build the image, step by step — and Docker builds an **image** from it (a static, read-only package), which you then **run** as a **container** (a live, running instance of that image).

Kubernetes (often abbreviated “K8s” — K, 8 letters, S) is a system for running many containers across many machines, automatically. It handles things a human shouldn’t have to do by hand: if a container crashes, Kubernetes restarts it; if you need 3 copies of your app running for reliability, Kubernetes keeps 3 running even if one dies; if a node (a physical or virtual machine in the cluster) needs to be taken down for maintenance, Kubernetes moves your containers off it first. You describe the *desired state* (“I want 3 copies of this container running, each with these resource limits, each exposed on this port”) in YAML files, and Kubernetes continuously works to make reality match that description.

2.2 What was built: a real, working service

To make this concrete instead of theoretical, a small real application was built: `app/status-service/`, a Flask (Python web framework) app that reads this repo’s `STATUS.md` file and serves it as a web

page and as JSON, plus two special endpoints used by Kubernetes itself:

- **/healthz** — a **liveness probe** endpoint. Kubernetes calls this periodically to ask “are you still alive?” If it stops responding, Kubernetes assumes the container is stuck or dead and restarts it.
- **/readyz** — a **readiness probe** endpoint. This is subtly different from liveness: it asks “are you *ready to serve traffic right now?*” A container can be alive but not ready (e.g., still loading data on startup) — Kubernetes won’t send it real user traffic until readiness passes, even though it wouldn’t restart it either.

This distinction — alive vs. ready — is a classic thing worth being able to explain clearly in an interview: liveness failing means “restart me,” readiness failing means “don’t send me traffic yet, but don’t restart me either.”

2.3 The Dockerfile: hardening a container image

Open `app/status-service/Dockerfile`. A few choices worth understanding:

- **Multi-stage build.** The Dockerfile has two stages: a “builder” stage that installs build tools and dependencies, and a final “runtime” stage that copies only the finished, installed dependencies out of the builder — leaving all the build tools behind. This means the final image that actually runs in production is smaller and has fewer tools an attacker could abuse if they got inside it (no compilers, no package managers sitting around).
- **Non-root user.** By default, a container runs as **root** — the most privileged user on Linux. If an attacker manages to break out of the application and into the container’s shell, running as root gives them far more they could potentially do (and a small chance of breaking further out of the container itself). This Dockerfile creates a dedicated, unprivileged user (uid/gid 10001) and runs the application as that user instead.
- **No unnecessary packages.** The runtime image doesn’t install anything beyond exactly what’s needed to run a Python/Flask app under gunicorn (a production-grade WSGI server — Flask’s own built-in server is meant for development only). Every extra package in an image is one more thing that could have its own vulnerability.
- **HEALTHCHECK** — Docker’s own, container-level version of the liveness probe idea, independent of Kubernetes, useful even if you’re just running the container directly.

You were told earlier that this Dockerfile was actually built and run (`docker build` succeeded, and it was run with `--read-only --tmpfs /tmp --user 10001:10001` to prove it works under the same restrictions Kubernetes will later impose). That test caught a real, concrete lesson worth remembering: gunicorn needs *somewhere* writable, even in an otherwise fully locked-down container, and the first attempt (before adding a writable `/tmp`) failed with `FileNotFoundError: No usable temporary directory`. Hardening isn’t a checklist you apply blindly — it’s a negotiation between “as locked down as possible” and “the software still has to actually run,” and you find that boundary by testing, not by guessing.

2.4 The Kubernetes manifests: `k8s/`

```
k8s/
|-- base/
|   |-- deployment.yaml      # "run N copies of this container, like this"
|   |-- service.yaml         # "here's a stable network address for it"
```

```
| |-- networkpolicy.yaml    # "here's exactly what network traffic is allowed"
| |-- pdb.yaml             # "never take down all copies at once"
| `-- kustomization.yaml   # ties the base files together
|-- overlays/
| |-- dev/                 # 1 replica, a moving "demo" image tag
| `-- prod/                # 3 replicas, a pinned image tag
```

This uses **Kustomize**, a tool built into `kubectl` itself, for managing environment-specific differences without duplicating the whole YAML file per environment — exactly the same “write once, reuse with different inputs” idea as the Terraform modules in Part 1. The `base/` directory has the common definition; each `overlays/` directory is a small patch describing only what’s *different* for that environment (how many replicas, which image tag).

`deployment.yaml` is the most important file to understand, because it’s where most of the Kubernetes-specific hardening lives:

- **resources.requests / resources.limits** — tells Kubernetes how much CPU and memory this container needs at minimum (requests, used for scheduling — “don’t put this on a node that doesn’t have this much spare capacity”) and at maximum (limits — “kill or throttle it if it tries to use more than this”). Without limits, one misbehaving container can starve every other container on the same physical machine of resources — this is as much a reliability control as a security one.
- **securityContext.runAsNonRoot: true** — a second, independent enforcement of “don’t run as root,” this time at the Kubernetes layer, not just the Dockerfile layer. Defense in depth means not trusting a single layer to get it right.
- **readOnlyRootFilesystem: true** — the container’s filesystem can’t be written to at all, except for the one path explicitly given a writable volume (here, an `emptyDir` volume mounted at `/tmp`, tying back to the unicorn lesson from 2.3). This limits what an attacker could do even if they got code execution inside the container — they can’t drop a malicious script onto disk and run it, because there’s (almost) nowhere to write.
- **capabilities.drop: ["ALL"]** — Linux capabilities are fine-grained permissions beyond simple root/non-root (things like “can bind to a low-numbered network port,” “can change file ownership”). Dropping all of them means the container has none of these extra powers unless explicitly re-added — the same least-privilege instinct as the AWS IAM policy in Part 1.
- **allowPrivilegeEscalation: false** — prevents a process inside the container from gaining more privileges than it started with (for example, via a `setuid` binary).
- **seccompProfile: RuntimeDefault** — seccomp filters which raw Linux system calls a process is even allowed to make; `RuntimeDefault` applies a sane, restrictive default set rather than allowing everything the kernel supports.

Together, these implement what Kubernetes calls the “**restricted**” **Pod Security Standard** — the strictest of Kubernetes’ three built-in security profiles (privileged / baseline / restricted). Being able to say “this deployment follows the restricted Pod Security Standard, here’s each control and why” is a genuinely strong, specific thing to say in an interview.

networkpolicy.yaml — by default, every pod in a Kubernetes cluster can talk to every other pod, on any port. A `NetworkPolicy` is how you turn that off. This repo uses three separate policy objects implementing **default-deny**: first, deny all traffic; then, explicitly allow only inbound traffic on port 8080 (what the app actually needs); then, explicitly allow only outbound DNS traffic (needed for the app to resolve hostnames at all — everything else outbound is blocked). This is the network

equivalent of the IAM least-privilege pattern from Part 1: default to nothing, then explicitly allow only what’s actually needed.

pdb.yaml — a **PodDisruptionBudget**, set to `minAvailable: 1`, paired with the 3-replica production overlay. This protects against *voluntary* disruption — for example, if a cluster administrator is draining a node for maintenance or doing a rolling upgrade, Kubernetes will respect this budget and won’t take down so many copies at once that fewer than 1 remains available. This is a different concern from a crash (which liveness probes handle) — it’s about planned maintenance not becoming an unplanned outage.

2.5 What was, and wasn’t, verified

Docker was available, so the image was actually built and actually run under the restricted settings above — that’s real, tested evidence, not just a YAML file that looks right on paper. There was no real Kubernetes cluster available (no `kubectl`, no `kind`), so the manifests themselves were validated two other ways: every YAML file was parsed to confirm it’s syntactically valid, and `kustomize build` was run against both overlays to confirm they render into valid, complete Kubernetes objects without errors (this actually caught one real issue — a deprecated `commonLabels` field that needed updating to `labels` in newer Kustomize versions).

The honest gap: none of this has been proven to actually schedule, run, self-heal, or serve real traffic on a live cluster. Writing a correct manifest and watching Kubernetes actually enforce it are different skills, and only the first one is demonstrated here. Standing up a free local cluster (`kind` or `minikube`) and running `kubectl apply -k k8s/overlays/dev` against it is the natural next exercise — see Part 6.

Part 3 — CI/CD pipelines with security scanning built in

3.1 What problem does this solve?

Without an automated pipeline, “does this code work, and is it safe to ship” depends on a human remembering to run the right checks by hand, every single time, before every single change. Humans forget, get rushed, or skip steps under deadline pressure. A **CI/CD pipeline** automates this entirely: every time code is pushed, a defined sequence of checks runs automatically, with no human able to accidentally skip a step.

- **CI (Continuous Integration)** is the practice of automatically building and testing every change, so problems are caught within minutes of being introduced, not weeks later when someone else’s change conflicts with yours.
- **CD (Continuous Delivery/Deployment)** extends this to automatically deploying changes that pass all checks — Delivery means it’s automatically *ready* to deploy (a human clicks a button), Deployment means it goes out *automatically* with no human step at all. This repo’s pipeline stops at CI plus security gating; it doesn’t auto-deploy anywhere, because there’s no real environment to deploy to yet.

The DevSecOps piece is folding security scanning directly into this same automated pipeline, as gates that can *block* a merge — not a separate, optional, easily-skipped step that happens later.

3.2 What was built: `.github/workflows/`

Two workflow files were written for **GitHub Actions** (GitHub’s built-in CI/CD system — you write YAML files describing “jobs,” and GitHub runs them on its own servers every time code is pushed):

- **ci.yml** — the “does it build and pass basic checks” pipeline: lints the Python code with **ruff** (a linter — a tool that checks code for style issues and common mistakes without actually running it), and runs any unit tests that exist.
- **security.yml** — the DevSecOps scanning pipeline, described stage by stage below. This one also runs on a weekly schedule (**cron**), in addition to every push — because a dependency that was safe last week might have a newly disclosed vulnerability *this* week, even though nothing in the code itself changed. Security scanning isn’t a one-time gate at commit time; known-vulnerability databases update constantly, so scanning needs to repeat on a schedule too.

3.3 Walking through the security stages

Read these in the order they run:

1. **Secrets detection (gitleaks)**. This scans for things that look like passwords, API keys, or private keys accidentally committed to the repository. Crucially, it scans the *full git history*, not just today’s changes — because a secret committed three commits ago and later “removed” is still sitting in git history, retrievable by anyone with read access to the repo, unless it’s rotated (changed at the source) or the history is rewritten. This is why secret scanning has to run against history, and why the real fix for a leaked secret is always “rotate it immediately,” not “delete the line and commit again.”
2. **SAST — Static Application Security Testing (bandit)**. “Static” means it analyzes source code *without running it*, looking for known-dangerous patterns (e.g., building a shell command by concatenating strings from user input, which risks command injection; using a weak/broken cryptographic function; hardcoding credentials in code). This is different from a linter — a linter cares about style and correctness, a SAST tool specifically cares about security-relevant patterns.
3. **SBOM — Software Bill of Materials (syft), and dependency scanning (pip-audit)**. An SBOM is a complete, machine-readable inventory of every library your software depends on, and their exact versions — think of it as an ingredient list. It matters because when a new vulnerability is announced in some library next month, you need to be able to answer “do we use that library, and where?” in minutes, not by grepping every repo by hand. **pip-audit** is the actual vulnerability check: it takes your dependency list and checks it against a database of known vulnerabilities (CVEs — Common Vulnerabilities and Exposures, the standard naming scheme for publicly disclosed security flaws).
4. **Container image scanning (trivy)**. This is subtly different from #3: it scans the *built container image itself*, including the base OS image and every layer, not just your application’s direct Python dependencies. A vulnerability could exist in a system library baked into the base image that your **requirements.txt** never mentions at all. The pipeline builds the actual image and fails if **trivy** finds any HIGH or CRITICAL severity vulnerability that has a known fix available.
5. **IaC scanning (tfsec)**. This scans the Terraform code from Part 1 for security misconfigurations — the same category of check as SAST, but aimed at infrastructure definitions

instead of application code. It's what would have caught the Azure provider attribute mistake mentioned in Part 1 automatically, on every push, rather than relying on a human catching it by hand-reading.

6. **Kubernetes manifest scanning (kube-linter).** Same idea again, aimed at the Kubernetes YAML from Part 2 — checking for things like missing resource limits, running as root, or missing readiness probes, as a machine-enforced backstop in case a future change to those manifests quietly drops one of the hardening choices explained in Part 2.

Notice the *shape* here: five different tools, each aimed at a different layer (secrets, application code, dependencies, container images, infrastructure code, deployment manifests), all wired into the same pipeline. This layered approach is usually called **defense in depth** — no single scanner catches everything, so you stack several, each covering a different class of mistake.

3.4 Pipeline design choices worth naming in an interview

- **Fail-fast staging.** Cheap, fast checks (lint, secrets detection) run before expensive, slow ones (building and scanning a container image). If the cheap check fails, you find out in seconds instead of waiting minutes for a container build that was never going to matter anyway.
- **Least-privilege pipeline permissions.** Each GitHub Actions job declares exactly what it's allowed to do (`permissions:` `contents:` `read` by default, with `security-events:` `write` only granted to the specific jobs that need to upload scan results). This is the same least-privilege instinct as the AWS IAM role in Part 1, applied to the CI system's own credentials — if a job's token leaked, the damage it could do is limited to exactly what that job actually needed.
- **Pinned action versions.** Every third-party GitHub Action used is pinned to a specific version tag, not floating on something like `@main` or `@latest`. An unpinned dependency in your CI pipeline is a supply-chain risk: if that action's repository is compromised and pushes malicious code to its `main` branch, every pipeline using `@main` picks it up automatically, silently, on the very next run.
- **Centralized findings (SARIF).** Several of these tools output their results in a standard format called **SARIF** (Static Analysis Results Interchange Format), which gets uploaded to GitHub's built-in “code scanning” dashboard. Instead of digging through seven different tools' log output to find what failed, everything lands in one place.
- **Graceful incremental rollout.** Every security job first checks whether the file or directory it needs actually exists yet (`app/status-service/Dockerfile`, `infra/terraform/`, `k8s/`) and skips cleanly with a note if not, rather than hard failing. This matters in real organizations where a platform pipeline often gets rolled out before every team has finished migrating onto it — the pipeline shouldn't break for teams that haven't adopted a given piece yet.

3.5 The GitLab mirror: `.gitlab-ci.yml`

The job posting calls out GitLab specifically (alongside Jenkins and Azure DevOps) as an example CI/CD tool. Since this repo lives on GitHub, a second file, `.gitlab-ci.yml`, was written that reproduces the exact same stages in GitLab's own pipeline syntax — same tools, same gating logic, same “skip cleanly if the target doesn't exist yet” pattern, just translated into GitLab's dialect (`stages:`, `rules:` `exists:` instead of GitHub's `if:` conditions on a step output, GitLab's built-in `artifacts:` blocks instead of GitHub Action's `upload-artifact`). It's only been checked for valid YAML syntax so far — it has never actually run, because that needs a real GitLab project and a runner (see Part 6 for how to do that cheaply). The lesson worth taking from this exercise, once

you do run it: the *concepts* (stages, jobs, artifacts, conditional execution, least-privilege tokens) are identical across every CI system. Only the YAML dialect changes. Once you’ve built one pipeline well, learning a second CI system is mostly a vocabulary exercise, not a from-scratch relearning.

Part 4 — Where the “sysadmin” and “compliance” pieces of the job fit

The job description also asks for Windows Server administration, Active Directory, Group Policy Objects, endpoint management (MECM/SCCM, Intune), and CIS hardening baselines. None of that has a natural home inside a git repository the way Terraform or Kubernetes manifests do — those things require actual Windows machines, an actual domain, and an actual endpoint fleet to practice against. Rather than pretend otherwise, that gap is named honestly in Part 5, with concrete, low-cost suggestions for closing it.

What *can* be said here is how these pieces connect to everything above, because that connection is something you can explain in an interview even without having built it yet:

- **Active Directory** is the identity system — the “who is this person, what groups are they in, what are they allowed to do” database — that a real enterprise would use to control who can push code, who can approve a Terraform change, who can access the Kubernetes cluster. The Azure Terraform module in Part 1 has a comment block sketching exactly where this attaches: Azure AD Connect syncs an on-prem AD to Azure AD (now branded “Microsoft Entra ID”), and from there, group membership in AD can control access to the cloud resources this repo provisions.
 - **Group Policy Objects (GPOs)** are how AD pushes configuration and security settings out to every Windows machine in a domain automatically — the conceptual sibling of “Infrastructure as Code,” but for endpoint configuration instead of cloud resources: a declared desired state, enforced automatically, instead of a human configuring each machine by hand.
 - **MECM/SCCM and Intune** are Microsoft’s endpoint management tools — MECM/SCCM (Microsoft Endpoint Configuration Manager / System Center Configuration Manager) traditionally for on-prem-managed Windows devices, Intune for cloud-managed devices (including mobile). They’re the tools that actually push software updates, enforce security policy, and manage device compliance at scale — the endpoint-level enforcement arm of the same “hardening baseline” idea that shows up as encrypted S3 buckets and non-root containers everywhere else in this document.
 - **CIS Benchmarks** (Center for Internet Security) are industry-standard, detailed hardening checklists for specific systems — there’s one for Windows Server, one for a Kubernetes cluster, one for AWS, one for Ubuntu, and so on. Every hardening choice made in Parts 1–3 (encryption at rest, non-root containers, least-privilege IAM, default-deny networking) is *CIS-aligned in spirit*, but none of it has been checked against an actual, scored CIS Benchmark using a real scanner (like **kube-bench** for Kubernetes, or AWS Config conformance packs for AWS) — because that requires a live account or cluster to score, not just source code.
-

Part 5 — Honest gaps: what this repo cannot teach you

Being able to say precisely what you *haven't* proven yet, and why, is a mark of seniority, not a weakness. Here's the honest list:

1. **Windows Server, Active Directory, GPOs, MECM/SCCM, Intune** — nothing here. These require real (or lab) Windows infrastructure. The single most valuable next step for closing this gap: spin up a small home lab — even a couple of VMs (VirtualBox, Hyper-V, or a free-tier cloud VM) running Windows Server with the Active Directory Domain Services role installed, join a second VM to that domain, and practice creating users, groups, and a GPO that pushes a real setting. Microsoft also offers free trial access to Intune for exactly this kind of self-study.
 2. **Everything in Parts 1–3, run against a real environment.** The Terraform has never been **apply**'d to a live AWS/Azure account. The Kubernetes manifests have never been applied to a live cluster. The CI pipelines have been syntax-checked but never actually executed by a real GitHub Actions runner or GitLab runner. Writing correct infrastructure-as-code and *operating* it under real conditions (real IAM federation, real cluster autoscaling, real on-call pages at 2am) are different skills, and this repo only demonstrates the first one.
 3. **Formal CIS Benchmark scoring.** The hardening choices made are CIS-*aligned*, but nothing here produces an actual scored compliance report the way **kube-bench**, AWS Config conformance packs, or Azure Policy's CIS initiative would against a live account or cluster.
 4. **Coder** (a managed development-environment platform, named specifically in the job's required qualifications). Nothing was built here — Coder itself deploys via a Helm chart onto Kubernetes, so the hardening patterns from Part 2 (NetworkPolicy, restricted Pod Security Standard, resource limits) are directly reusable as the security baseline for a Coder workspace template once you do stand one up.
 5. **GitLab, for real.** `.gitlab-ci.yml` exists and is syntactically valid, but has never actually run against a real GitLab project. See Part 6 for exactly how to close this one cheaply — it's the lowest-effort gap on this list to close.
-

Part 6 — Suggested next exercises, roughly in order of effort

1. **Run the GitLab pipeline for real.** Create a free account at `gitlab.com`, create a new project, and add it as a second git remote alongside GitHub: `git remote add gitlab git@gitlab.com:<you>/poetic-musings.git`, then `git push gitlab main`. GitLab auto-detects `.gitlab-ci.yml` and runs it. Watch it run, read any failures, fix them. This is a few minutes of setup for a real, working example of a second CI system on your resume.
2. **Stand up a local Kubernetes cluster.** Install `kind` (Kubernetes in Docker — a tool that runs a real, small Kubernetes cluster entirely inside Docker containers on your own machine, free, no cloud account needed) and run `kubectl apply -k k8s/overlays/dev` against it. Watch the pod actually schedule, hit `/healthz`, and try deliberately breaking a security setting (e.g., remove `readOnlyRootFilesystem`) to see what actually changes.
3. **Install terraform, tfsec, and checkov locally**, run `terraform validate` and `terraform plan` (plan only — don't **apply** against a real account unless you actually want to provision and pay for real cloud resources) against `infra/terraform/`, and fix any findings the scanners report.

4. **Build the Windows AD home lab** described in Part 5 — this is the highest-value, highest-effort gap to close, and the one most worth scheduling real time for.
 5. **Study for one certification** relevant to the preferred qualifications — AWS Certified DevOps Engineer, CKA (Certified Kubernetes Administrator), or CompTIA Security+ are all reasonable starting points and each has well-trodden, inexpensive study paths.
-

Glossary

CI/CD — Continuous Integration / Continuous Delivery (or Deployment): automatically building, testing, and (for Deployment) releasing code on every change.

CIS Benchmark — an industry-standard, detailed hardening checklist for a specific system (Windows Server, Kubernetes, AWS, etc.), published by the Center for Internet Security.

Container — a packaged, portable unit of software including everything it needs to run, so it behaves identically across machines.

CVE — Common Vulnerabilities and Exposures: the standard public naming/numbering system for disclosed security vulnerabilities.

DevSecOps — DevOps (collapsing the wall between building and running software) with security scanning built directly into the automated pipeline, rather than handled as a separate, later review.

Docker — the most common tool for building and running containers.

IaC (Infrastructure as Code) — describing infrastructure in version-controlled text files instead of manual cloud-console clicks, so it's repeatable, reviewable, and recreatable.

IAM (Identity and Access Management) — the AWS system (and general concept) for controlling who/what can do what to which resources.

IDP (Internal Developer Platform) — the shared infrastructure, pipelines, and tooling a platform team builds for every other team to build on.

Kubernetes (K8s) — a system for running and automatically managing many containers across many machines.

Least privilege — granting exactly the access needed and no more, so a compromised credential or component causes the smallest possible amount of damage.

Module (Terraform) — a reusable, parameterized package of infrastructure resources.

Pod Security Standards — Kubernetes' three built-in security profiles for pods: privileged, baseline, and restricted (strictest).

SARIF — Static Analysis Results Interchange Format: a standard format for security scan results, so different tools' findings can be viewed in one place.

SAST (Static Application Security Testing) — analyzing source code for security-relevant patterns without running it.

SBOM (Software Bill of Materials) — a complete, machine-readable inventory of every dependency a piece of software uses.

Shift left — moving security checks earlier in the development timeline (into the automated pipeline, at commit time) instead of as a late, pre-launch review.

State (Terraform) — Terraform's record of what infrastructure it has already created, used to compute what needs to change on the next run.

Terraform — the most widely used Infrastructure-as-Code tool, using the HCL language.

This document was built inside the poetic-musings repository as a hands-on training exercise. See `infra/terraform/`, `app/status-service/`, `k8s/`, `.github/workflows/`, `.gitlab-ci.yml`, and `SECURITY.md` for the actual code this document walks through.